

1. What is Linux Disk Management?

Linux Disk Management is the process of **organizing, creating, modifying, and controlling storage devices (hard disks, SSDs, partitions, filesystems, swap, and logical volumes)** in a Linux system so that the operating system and applications can **store and retrieve data efficiently**.

In Linux, disks are managed through **block devices** located in the `/dev` directory.

Example:

`/dev/sda` → entire disk
`/dev/sda1` → partition 1
`/dev/sdb` → second disk

Disk management involves tasks like:

- Detecting new disks
- Creating partitions
- Creating filesystems
- Mounting filesystems
- Managing storage using LVM
- Managing swap space
- Monitoring disk usage

So in simple terms:

Disk management = controlling how storage space is organized and used in Linux.

1.1 Why Disk Management is Important

Disk management is important because **Linux cannot use a disk directly until it is properly structured.**

A new disk must go through several steps before it can store files:

Disk → Partition → Filesystem → Mount → Usable Storage

Without disk management:

- The system cannot store data
 - Applications cannot write logs or databases
 - The OS cannot boot properly
 - Storage cannot be expanded or controlled
-

1.2 Why We Need Disk Management

We need disk management for several important reasons.

1.2.1 Organizing Storage

A single disk can be divided into **multiple partitions** for better organization.

Example:

/dev/sda1 → /boot

/dev/sda2 → /

/dev/sda3 → /home

Benefits:

- Separate system files
 - Separate user data
 - Prevent system crashes due to full disk
-

1.2.2 System Stability

If everything is stored in one partition and it becomes **full**, the system may stop working.

Example:

If **/var** (logs) fills the root partition:

System may crash

Services may stop

Solution:

/var as separate partition

This prevents logs from filling the entire system.

1.2.3 Better Security

Partitions allow **security separation**.

Example:

/home → user files

/tmp → temporary files

/var/log → logs

You can apply:

- different permissions
 - mount restrictions
 - encryption
-

1.2.4 Efficient Storage Usage

Linux administrators must **allocate disk space efficiently**.

Example:

A database server may need:

/var/lib/mysql → large storage

/logs → separate disk

/backups → separate disk

Disk management ensures **correct storage allocation**.

1.2.5 Performance Optimization

Some applications perform better when storage is separated.

Example:

Database data → disk 1

Database logs → disk 2

OS → disk 3

This reduces disk I/O bottlenecks.

1.2.6 Flexibility with LVM

With **Logical Volume Management (LVM)** we can:

- extend disks
- shrink disks
- combine multiple disks
- take snapshots

Example:

Disk1 + Disk2 → Volume Group

Volume Group → Logical Volume

If space is low:

Add new disk → extend logical volume

No downtime required.

1.2.7 System Recovery and Backup

Separate partitions make **backup and recovery easier**.

Example:

/home partition backup only

If the OS fails:

Reinstall OS

Mount /home

Data remains safe

1.3 Why Linux Uses Mounting

Linux uses a **single directory tree** for everything.

Instead of using drive letters like Windows:

C:

D:

E:

Linux attaches storage to directories.

Example:

/mnt/data

/home

/backup

This process is called **mounting**.

Example:

mount /dev/sdb1 /data

Now **/data** contains the disk contents.

1.4 Role of Partitioning in Disk Management

Partitioning divides a disk into smaller logical sections.

Example:

500GB disk

Can be divided into:

100GB → OS

200GB → Applications

200GB → Data

Benefits:

- isolation
 - management
 - stability
-

1.5 What Happens When a New Disk is Added

When a new disk is added to Linux, it goes through this process.

Step 1: Detect disk

lsblk

fdisk -l

Step 2: Create partition

fdisk /dev/sdb

Step 3: Inform kernel about changes

partprobe

Step 4: Create filesystem

mkfs.xfs /dev/sdb1

Step 5: Create mount point

```
mkdir /data
```

Step 6: Mount disk

```
mount /dev/sdb1 /data
```

Step 7: Make it permanent

Edit:

```
/etc/fstab
```

1.6 What is **partprobe** and Why It Is Used

When we create or modify partitions using tools like:

```
fdisk  
parted  
gdisk
```

the **kernel may still use the old partition table**.

To update the kernel without rebooting, we use:

```
partprobe
```

1.6.1 What **partprobe** does

It tells the **Linux kernel to re-read the partition table**.

Example workflow:

```
fdisk /dev/sdb  
(create partition)
```

```
partprobe
```

Now the kernel detects:

```
/dev/sdb1
```

Without **partprobe**, sometimes you must **reboot the system**.

1.7 Simple Real-Life Example

Think of a disk like a **large building**.

Building = Disk

Rooms = Partitions

Furniture = Files

Room door = Mount point

Disk management decides:

- how many rooms exist
- how big each room is
- what each room is used for

1.8 Summary

Linux disk management is used to:

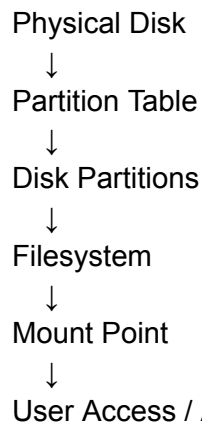
- organize storage
- create partitions
- create filesystems
- mount storage
- manage swap
- manage LVM
- control disk usage
- improve system performance and stability

And commands like **partprobe** help the system **recognize new partitions without rebooting**.

2. Linux Storage Architecture Overview

Linux storage works in **multiple layers**, where each layer performs a specific function to allow users and applications to store and access data.

The architecture looks like this:



Each layer is explained below.

2.1. Physical Disk (Hardware Layer)

A **physical disk** is the actual storage hardware connected to the system.

Examples:

- HDD (Hard Disk Drive)
- SSD (Solid State Drive)
- NVMe Drive
- External USB Drive

- Cloud Block Storage (AWS EBS, Azure Disk)

In Linux, disks are represented as **block devices**.

Location:

/dev

Examples:

/dev/sda

/dev/sdb

/dev/nvme0n1

Explanation:

Device	Meaning
/dev/sda	First disk
/dev/sdb	Second disk
/dev/sdc	Third disk
/dev/nvme0n1	NVMe SSD

Example system:

/dev/sda → 500GB Disk

/dev/sdb → 1TB Disk

To view disks:

lsblk

Example output:

NAME	SIZE	TYPE
sda	100G	disk
sdb	500G	disk

At this stage, the disk is **empty and unusable**.

We must **create partitions**.

2.2. Partition Table

The **partition table** is a structure stored on the disk that defines how the disk is divided.

It tells Linux:

- where partitions start
- where partitions end
- partition type

Think of a partition table like a **table of contents of a book**.

Two main types exist:

Under the **Partition Table layer**, two major standards are used to organize partitions on a disk:

- **MBR (Master Boot Record)**
- **GPT (GUID Partition Table)**

These define **how the disk is structured and how partitions are stored and identified**. Let's examine them in detail.

2.2.1. MBR (Master Boot Record)

a. What is MBR?

MBR is the **oldest partitioning scheme** used in PCs.

It was introduced in **1983 with IBM PC DOS** and remained the standard for many years.

MBR stores:

1. **Bootloader code**
2. **Partition table**

3. Boot signature

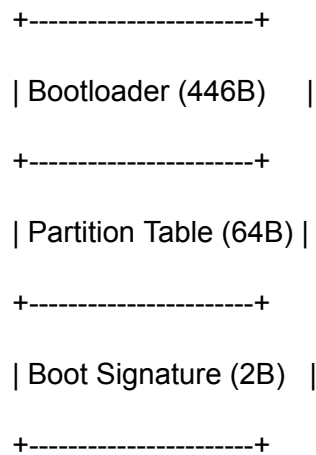
All of this is stored in the **first sector of the disk (512 bytes)**.

b. Structure of MBR

The first **512 bytes of the disk** contain:

Size	Component	Purpose
446 bytes	Bootloader	Code to start OS
64 bytes	Partition Table	Information about partitions
2 bytes	Boot Signature	Marks valid MBR

Diagram:



c. MBR Partition Table

The partition table stores **4 entries**, each representing a partition.

Each entry contains:

- Partition start location
- Partition size
- Partition type
- Boot flag

Example disk:

Disk (/dev/sda)

|— /dev/sda1

|— /dev/sda2

|— /dev/sda3

└— /dev/sda4

MBR allows **maximum 4 primary partitions**.

d. Extended and Logical Partitions

To overcome the 4-partition limit, **extended partitions** were introduced.

Structure:

Disk

|— Primary Partition

|— Primary Partition

|— Primary Partition

└— Extended Partition

└─ Logical Partition

└─ Logical Partition

└─ Logical Partition

Example:

/dev/sda1 → primary

/dev/sda2 → primary

/dev/sda3 → primary

/dev/sda4 → extended

/dev/sda5 → logical

/dev/sda6 → logical

e. MBR Limitations

i. Disk Size Limit

Maximum disk size supported:

2 TB

Reason: MBR uses **32-bit addressing**.

ii. Partition Limit

Maximum:

4 primary partitions

iii. No Redundancy

The partition table exists **only in one location**.

If the MBR gets corrupted:

- System may not boot
 - Partitions may become inaccessible
-

f. When MBR is Used

MBR is still used for:

- Older BIOS systems
 - Legacy operating systems
 - Small disks (< 2TB)
-

2.2.2. GPT (GUID Partition Table)

a. What is GPT?

GPT is the **modern partitioning system** that replaces MBR.

It is part of the **UEFI (Unified Extensible Firmware Interface)** standard.

GPT solves many limitations of MBR.

b. Key Features of GPT

Feature	GPT
Maximum disk size	~9.4 Zettabytes
Partitions allowed	128 (default)

Redundancy Yes

Partition identification GUID

Boot system UEFI

c. GPT Disk Layout

GPT stores partition information in **multiple locations**.

Structure:

```
+-----+
| Protective MBR   |
+-----+
| GPT Header       |
+-----+
| Partition Table  |
+-----+
| Disk Data        |
+-----+
| Backup Partition |
| Table + Header   |
+-----+
```

Important concept:

GPT keeps a **backup copy at the end of the disk**.

This makes it more reliable.

d. GPT Partition Table

GPT supports **128 partitions by default**.

Example:

/dev/sda1

/dev/sda2

/dev/sda3

/dev/sda4

/dev/sda5

/dev/sda6

...

No need for **extended or logical partitions**.

All partitions are treated the same.

e. GUID (Globally Unique Identifier)

Each partition in GPT has a **GUID**.

Example:

EBD0A0A2-B9E5-4433-87C0-68B6B72699C7

This uniquely identifies the partition.

Benefits:

- Prevents conflicts
 - Allows better system management
-

f. GPT Reliability

GPT improves reliability using:

i. Backup Partition Table

Stored at **end of disk**.

If main table fails, backup can restore it.

ii. CRC32 Checksums

GPT verifies data integrity using checksums.

g. GPT in Linux

Most modern Linux systems use GPT.

Check partition type:

```
sudo parted -l
```

Example output:

Partition Table: gpt

h. MBR vs GPT Comparison

Feature	MBR	GPT
Maximum disk size	2TB	9.4ZB
Partitions	4 primary	128

Partition structure	Primary + Extended	All primary
Data protection	None	Backup table
Boot system	BIOS	UEFI
Reliability	Low	High

i. Visual Comparison

i. MBR Disk

Disk

|— sda1

|— sda2

|— sda3

└— sda4

Extended example:

Disk

|— sda1

|— sda2

|— sda3

└— sda4 (extended)

|— sda5

|— sda6

└─ sda7

ii. GPT Disk

Disk

|─ sda1

|─ sda2

|─ sda3

|─ sda4

|─ sda5

|─ sda6

└─ sda7

No extended partitions needed.

j. How to Create GPT in Linux

Create GPT label:

```
sudo parted /dev/sdb
```

Then:

```
mklabel gpt
```

Create partition:

```
mkpart primary ext4 1MiB 10GiB
```

k. How to Check Disk Type

Using **parted**:

sudo parted -l

Example output:

Disk /dev/sda

Partition Table: gpt

or

Partition Table: msdos

msdos means **MBR**.

2.3. Disk Partitions

A **partition** is a logical section of a disk.

It divides one disk into multiple smaller disks.

Example:

One disk:

/dev/sda (500GB)

Partitions:

/dev/sda1 100GB

/dev/sda2 200GB

/dev/sda3 200GB

Why partitions are used:

Reason	Explanation
Separate OS files	Root partition
User data	/home
System logs	/var
Boot files	/boot

Swap memory swap

Example real Linux partition layout:

```
/dev/sda1 → /boot
/dev/sda2 → /
/dev/sda3 → /home
/dev/sda4 → swap
```

View partitions:

lsblk

Example:

```
NAME  SIZE TYPE
sda   100G disk
├─sda1 1G part
├─sda2 50G part
└─sda3 49G part
```

2.4. Filesystem

A **filesystem** organizes data inside a partition.

Without a filesystem, Linux cannot store files.

Filesystem handles:

- file storage
- directory structure
- permissions
- metadata
- disk allocation

Common Linux filesystems:

Filesystem	Use
------------	-----

ext4	Most common
xfs	Enterprise servers
btrfs	Modern advanced filesystem
vfat	USB drives
ntfs	Windows disks

Example:

/dev/sda1 → ext4

/dev/sda2 → xfs

Create filesystem:

```
mkfs.ext4 /dev/sdb1
```

Example structure inside filesystem:

```
/
|
|— home
|— var
|— etc
|— usr
```

The filesystem manages:

- blocks
- inodes
- directory entries

2.5. Mount Point

A **mount point** is a directory where a filesystem is attached.

In Linux, everything is accessed through a **single directory tree**.

Example:

Mount a disk:

```
mount /dev/sdb1 /data
```

Now the disk appears at:

/data

Example structure:

```
/
├── bin
├── etc
├── home
└── data ← mounted disk
```

Files written in **/data** go to **/dev/sdb1**.

Check mounted disks:

```
df -h
```

Example:

Filesystem	Size	Used	Mounted on
/dev/sda1	50G	20G	/
/dev/sdb1	200G	10G	/data

2.6. User and Application Access

Applications and users interact only with **files and directories**, not directly with disks.

Example:

User command:

```
touch /data/file.txt
```

Flow of operation:

User Command



Filesystem



Disk Blocks

↓
Physical Disk

Application example:

Database writes data:

`/var/lib/mysql`

Linux stores it inside:

`/dev/sda2`

2.6.1 Complete Storage Flow Example

Suppose you add a new disk.

Step-by-step architecture:

Step 1 — Physical disk attached

`/dev/sdb`

Step 2 — Create partition

`/dev/sdb1`

Step 3 — Create filesystem

`mkfs.ext4 /dev/sdb1`

Step 4 — Mount filesystem

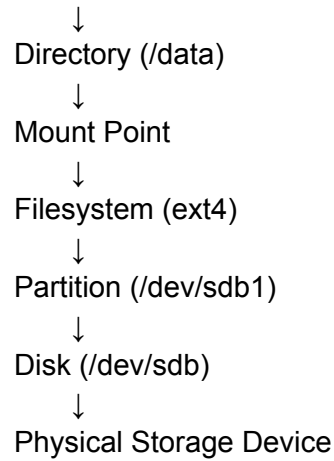
`mount /dev/sdb1 /backup`

Step 5 — Store data

`/backup/files`

2.6.2 Simple Visualization

User/Application



2.6.3 Why Linux Uses This Architecture

Benefits:

a. Flexibility

Multiple disks can be mounted anywhere.

Example:

/data
/logs
/backups

b. Security

Separate partitions prevent system crashes.

Example:

/var full → system still runs

c. Performance

Different filesystems optimized for workloads.

Example:

- XFS → databases
 - EXT4 → general usage
-

d. Scalability

Supports:

- RAID
 - LVM
 - Network storage
-

3. Disk Devices in Linux

3.1 What Are Disk Devices in Linux?

In Linux, **storage devices like hard disks, SSDs, USB drives, and NVMe drives are treated as special files called device files.**

These files are located in the directory:

`/dev`

So when Linux detects a disk, it creates a **device file inside `/dev`** that represents that disk.

Example:

```
/dev/sda  
/dev/sdb  
/dev/nvme0n1
```

These files allow the **operating system and applications to communicate with the hardware.**

3.2 Linux Philosophy

Linux follows a principle:

Everything in Linux is treated as a file.

This includes:

- disks
- printers
- USB devices
- terminals
- keyboards

Example device files:

/dev/sda
/dev/tty
/dev/null
/dev/random

3.3 What Are Block Devices?

Disk devices are known as **block devices**.

Block devices:

- store data in **fixed-size blocks**
- allow **random access to data**

Examples:

Device	Type
Hard disk	Block device

SSD Block device

USB drive Block device

Block devices are stored as:

`/dev/sdX`

3.4 Naming Convention of Disk Devices

Linux follows a naming pattern for disks.

3.4.1 SATA / SCSI / Virtual disks

`/dev/sdX`

Where:

Name	Meaning
------	---------

sd	SCSI disk (used for SATA also)
----	--------------------------------

a	first disk
---	------------

b	second disk
---	-------------

c	third disk
---	------------

Example:

`/dev/sda`

`/dev/sdb`

`/dev/sdc`

Example system:

`/dev/sda` → main OS disk

`/dev/sdb` → second disk

`/dev/sdc` → backup disk

3.5 Partitions of Disk Devices

Partitions are represented by **numbers added to disk names**.

Example:

```
/dev/sda1  
/dev/sda2  
/dev/sda3
```

Meaning:

Device	Description
/dev/sda	whole disk
/dev/sda 1	first partition
/dev/sda 2	second partition

Example disk layout:

```
/dev/sda  
├── /dev/sda1  
├── /dev/sda2  
└── /dev/sda3
```

3.6 Viewing Disk Devices & list information

Several commands can show disk devices.

3.6.1. lsblk

Displays all block devices.

lsblk

Example output:

NAME	SIZE	TYPE	MOUNTPOINT
sda	100G	disk	
└─sda1	1G	part	/boot

```
|_sda2 50G  part  /
|_sda3 49G  part  /home
```

Explanation:

Column	Meaning
NAME	device name
SIZE	disk size
TYPE	disk or partition
MOUNTPOINT	where it is mounted

3.6.2. fdisk -l

Shows detailed disk information.

```
sudo fdisk -l
```

Example:

```
Disk /dev/sda: 100 GiB
```

```
Disk /dev/sdb: 200 GiB
```

3.6.3. blkid

Shows filesystem UUID.

```
blkid
```

Example:

```
/dev/sda1: UUID="a1b2c3d4" TYPE="ext4"
```

3.6.4 df (Disk Free)

Shows **filesystem disk usage**.

```
df -h
```

Example:

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	50G	20G	28G	42%	/

Options:

Option	Purpose
df -h	Human readable
df -T	Show filesystem type
df -i	Show inode usage

3.6.5 du (Disk Usage)

Shows **directory space usage**.

```
du -sh /var/log
```

Example:

```
2.5G /var/log
```

Useful options:

```
du -sh *  
du -h --max-depth=1
```

3.7 How Linux Detects Disk Devices

When a disk is attached:

1. Hardware connects to system
2. Kernel detects device
3. Device driver loads

4. Kernel creates device file in `/dev`

Example process:

Hardware Disk
↓
Linux Kernel
↓
Device Driver
↓
`/dev/sda` created

3.8 When to Use Each Storage Type

Storage Type	When to Use	Example Use Cases
HDD	When you need very large storage at low cost and speed is not important	Backup servers, archives, media storage, surveillance storage
SSD	When you need good performance with reasonable cost	Operating systems, application servers, virtual machines
NVMe	When you need extremely high performance and very low latency	Databases, high-traffic websites, analytics workloads, AI/ML

4. Partition Management in Linux

4.1 What is a Partition?

A **partition** is a **logical division of a physical disk**.

It divides a single disk into multiple independent sections so that the operating system can manage storage efficiently.

Example:

A **500 GB disk** can be divided into multiple partitions.

Physical Disk (500GB)
`/dev/sda`

```
|
|— /dev/sda1 → 100GB
|— /dev/sda2 → 200GB
|— /dev/sda3 → 200GB
```

Each partition can have:

- its own **filesystem**
 - its own **mount point**
 - different **usage**
-

4.2 Why Partitions Are Used

Partitioning provides several benefits.

4.2.1. Data Organization

Partitions help organize data properly.

Example Linux layout:

```
/dev/sda1 → /boot
/dev/sda2 → /
/dev/sda3 → /home
/dev/sda4 → swap
```

Explanation:

Partition	Purpose
/boot	boot files
/	root filesystem
/home	user data
swap	virtual memory

4.2.2. Security and Stability

If one partition becomes full, it does not affect the others.

Example:

/var partition full

The system still works because / and /home are separate.

4.2.3. Performance

Separating workloads improves performance.

Example:

Database data → separate partition

Logs → separate partition

OS → separate partition

4.2.4. Easier Backup and Recovery

You can backup or restore partitions individually.

Example:

Backup only /home

4.3 Types of Partitions

Types of partitions are mainly defined in the **MBR partitioning scheme**. There are **three types of partitions**:

1. **Primary Partition**
2. **Extended Partition**
3. **Logical Partition**

Understanding these is important because they define **how a disk is divided and how many partitions can exist**.

4.3.1. Primary Partition

a. What is a Primary Partition?

A **primary partition** is a **normal partition directly created on a disk**.

It can contain:

- a **filesystem**
- an **operating system**
- **data**

Primary partitions are directly defined in the **partition table**.

Example disk:

Disk: /dev/sda

/dev/sda1

/dev/sda2

/dev/sda3

/dev/sda4

Each of these is a **primary partition**.

b. Limit of Primary Partitions

In **MBR partition tables**, you can create **maximum 4 primary partitions**.

Example:

Disk /dev/sda

Primary partitions:

sda1

sda2

sda3
sda4

After this, you **cannot create more partitions** unless you use an **extended partition**.

c. Usage of Primary Partitions

Primary partitions are usually used for **important system areas**.

Common usage in Linux:

Partition	Purpose
/dev/sda1	Boot partition
/dev/sda2	Root filesystem
/dev/sda3	Home directory
/dev/sda4	Swap space

Example layout:

/dev/sda1 → /boot
/dev/sda2 → /
/dev/sda3 → /home
/dev/sda4 → swap

4.3.2. Extended Partition

a. What is an Extended Partition?

An **extended partition** is a **special partition used as a container for logical partitions**.

It **does not store data directly**. Instead, it holds **logical partitions inside it**.

Why it exists:

MBR has a limit of **4 primary partitions**.

The extended partition was created to **overcome this limitation**.

b. Structure of Extended Partition

Example:

Disk /dev/sda

Primary partitions:

sda1

sda2

sda3

Extended partition:

sda4

├── sda5

├── sda6

└── sda7

Here:

sda4 → extended partition

sda5 → logical partition

sda6 → logical partition

sda7 → logical partition

c. Key Characteristics

Feature	Extended Partition
Purpose	Container for logical partitions
Number allowed	Only 1 extended partition per disk
Stores filesystem	No
Stores logical partitions	Yes

d. Why Extended Partition Is Needed

Without extended partitions:

Maximum partitions = 4

With extended partitions:

Unlimited logical partitions possible

Example:

```
sda1
sda2
sda3
sda4 (extended)
├── sda5
├── sda6
├── sda7
├── sda8
└── sda9
```

4.3.3. Logical Partition

a. What is a Logical Partition?

A **logical partition** is a partition created **inside an extended partition**.

Logical partitions behave like **normal partitions** and can store:

- filesystems
 - data
 - applications
-

b. Logical Partition Naming

Logical partitions **start from number 5**.

Example:

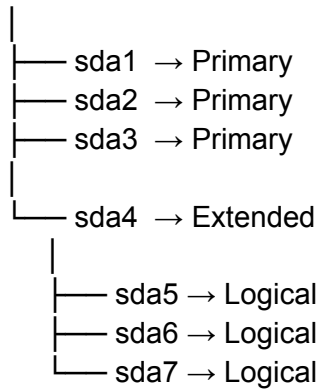
```
/dev/sda5
/dev/sda6
/dev/sda7
/dev/sda8
```

Why numbering starts from 5:

1–4 reserved for primary partitions
5+ used for logical partitions

c. Complete Example (Visual)

Disk: /dev/sda



4.4 Partition Numbering Example

Partition	Type
/dev/sda1	Primary
/dev/sda2	Primary
/dev/sda3	Primary
/dev/sda4	Extended
/dev/sda5	Logical
/dev/sda6	Logical
/dev/sda7	Logical

4.5 When Each Type Is Used

Partition Type	When to Use
Primary	For OS partitions like <code>/boot</code> or <code>/</code>

Extended	When more than 4 partitions are required
Logical	For additional data partitions

4.6 Important Note (GPT Disks)

If the disk uses **GPT partitioning**, then:

- There are **no primary/extended/logical partitions**
- All partitions are simply **standard partitions**
- Up to **128 partitions** are supported

Example:

```
/dev/sda1  
/dev/sda2  
/dev/sda3  
/dev/sda4  
/dev/sda5
```

No extended partition needed.

4.7 Quick Comparison

Feature	Primary	Extended	Logical
Maximum allowed	4	1	Many
Stores data	Yes	No	Yes
Contains other partitions	No	Yes	No
Location	Directly on disk	Directly on disk	Inside extended partition

4.8 Simple Analogy

Think of a disk like an **apartment building**.

Partition Type	Example
Primary partition	individual apartment
Extended partition	building floor
Logical partition	rooms on that floor

To know whether your Linux system disk is using **MBR or GPT partition table**, there are several commands you can use. These commands check the **partition table type stored on the disk**.

I'll explain the **most commonly used methods with examples**.

4.9 Identify Partition Table Type

To know whether your Linux system disk is using **MBR or GPT partition table**, there are several commands you can use. These commands check the **partition table type stored on the disk**.

I'll explain the **most commonly used methods with examples**.

4.9.1. Using **parted** (Most Clear Method)

This is one of the **best ways to check partition table type**.

Command

```
sudo parted -l
```

Example Output

```
Model: ATA VBOX HARDDISK
```

```
Disk /dev/sda: 107GB
```

```
Sector size (logical/physical): 512B/512B
```

Partition Table: gpt

Disk Flags:

Here:

Partition Table: gpt

Meaning:

Your disk is using **GPT partition table**.

If it is MBR

Example output:

Disk /dev/sda: 100GB

Partition Table: msdos

Important:

msdos = MBR partition table

4.9.2. Using fdisk

Another simple method.

Command

```
sudo fdisk -l
```

Example Output (GPT)

Disklabel type: gpt

Disk identifier: 9F0E9C20-9B9C-4F2A

Meaning:

Disklabel type: gpt

System is using **GPT**.

Example Output (MBR)

Disklabel type: dos

Meaning:

dos = MBR partition table

4.9.3. Using **lsblk**

lsblk can also show partition type.

Command

lsblk -f

However, this mostly shows **filesystem type**, not partition table directly. It is useful together with other commands.

Example:

```
NAME FSTYPE LABEL UUID MOUNTPOINT
```

```
sda
```

```
├─sda1 ext4
```

```
├─sda2 xfs
```

To know partition table, combine with **fdisk**.

4.10 Quick Trick to Remember

Output	Meaning
--------	---------

msdos / **MBR partition table**
dos

gpt **GPT partition table**

5. Partitioning Tools

5.1. fdisk (Fixed Disk Utility)

5.1.1 Overview

- **fdisk** is a **command-line utility for creating and managing disk partitions**.
- Works **best with MBR disks** (though recent fdisk versions also support GPT partially).
- It **modifies the partition table** of a disk.

fdisk -l Command – General Summary

The **fdisk -l** command is used to **list information about disks and their partitions** in a Linux system.

It displays two main sections:

1. **Disk Information**
2. **Partition Table Information**

1. Disk Information Fields

Field	Meaning
-------	---------

Disk	Disk device name (e.g., <code>/dev/sda</code>)
Size	Total storage capacity of the disk
Bytes	Disk size in bytes
Sectors	Total number of sectors on the disk
Disk model	Hardware or virtual disk model
Units	Measurement unit used (usually sectors)
Sector size	Logical and physical sector sizes
I/O size	Minimum and optimal read/write block size
Disklabel type	Partition table type (MBR or GPT)
Disk identifier	Unique identifier for the disk

2. Partition Table Fields

The partition table shows details about each partition.

Field	Meaning
-------	---------

Device	Partition name (e.g., <code>/dev/sda1</code>)
Boot	Indicates if the partition is bootable (*)
Start	Starting sector of the partition
End	Ending sector of the partition
Sectors	Total number of sectors allocated
Size	Partition size in KB/MB/GB
Id	Partition type code
Type	Partition type or filesystem category

3. Partition Type Codes (Examples)

Code	Meaning
83	Linux filesystem
82	Linux swap
8e	Linux LVM

07 NTFS

0b / 0c FAT32

5.1.2 When to Use fdisk

- Disk uses **MBR partition table**.
- Disk size is **less than 2TB** (MBR limitation).
- You want to **create, delete, or modify partitions** manually.

5.1.3 Basic fdisk Commands

1. Open a disk:

```
sudo fdisk /dev/sdb
```

2. Common commands inside fdisk:

Command	Purpose
---------	---------

m	Show help menu
---	----------------

p	Print current partition table
---	-------------------------------

n	Create new partition
---	----------------------

d	Delete partition
t	Change partition type (e.g., swap, Linux, etc.)
w	Write changes to disk
q	Quit without saving changes

5.1.4 Creating a Partition with fdisk

Step by step:

1. Start fdisk:

```
sudo fdisk /dev/sdb
```

2. Show current partitions:

p

3. Create new partition:

n

4. Select partition type:

p → primary

e → extended (for >4 partitions)

5. Choose partition number (1–4 for primary):

6. Specify start and end sectors:

Enter → start sector (default)

+20G → end sector (size of partition)

7. Write changes:

w

8. Partition created:

/dev/sdb1

5.1.5 Changing Partition Type

Example: Change a partition to **Linux swap**:

t

5 → partition number

82 → Linux swap type

w → write changes

5.1.6 Deleting a Partition

1. Delete a partition:

d

2. Specify partition number:

1

3. Write changes:

w

5.1.7 Viewing Partition Table

p

Example output:

Disk /dev/sdb: 100 GiB

Device	Boot	Start	End	Sectors	Size	Id	Type
--------	------	-------	-----	---------	------	----	------

/dev/sdb1		2048	41943039	41940992	20G	83	Linux
-----------	--	------	----------	----------	-----	----	-------

/dev/sdb2		41943040	83886079	41943040	20G	83	Linux
-----------	--	----------	----------	----------	-----	----	-------

- **Id** = partition type (83 = Linux, 82 = Swap)
- **Boot** = bootable partition (* marks bootable)

5.1.8 Advantages of fdisk

- Simple and widely available.
- Works well for **MBR disks**.
- Good for **small and medium disks**.

5.1.9 Disadvantages of fdisk

- Limited support for **GPT disks**.
- Command-line interface can be tricky for beginners.
- Cannot handle disks >2TB (MBR limitation).

5.2 parted (Partition Editor)

5.2.1 Overview

- **parted** is a modern disk partitioning tool.
- Works with **GPT and MBR disks**.
- Supports **large disks (>2TB)** and advanced features like **resizing partitions**.

5.2.2 When to Use parted

- Disk uses **GPT** (common in modern systems and UEFI).
- Disk size > **2TB**.
- You need **advanced operations**: create, resize, delete, format, move partitions.

5.2.3 Starting parted

`sudo parted /dev/sdb`

Example prompt:

(parted)

5.2.4 Key parted Commands

Command	Purpose
---------	---------

help	Show all commands
print	Show partitions and disk info
mklabel	Create new partition table (gpt or msdos)
mkpart	Create a new partition
rm	Delete a partition
resizepart	Resize a partition
quit	Exit parted

5.2.5 Creating Partition Table

MBR Example

mklabel msdos

GPT Example

mklabel gpt

- **msdos** = MBR
 - **gpt** = GUID Partition Table
-

5.2.6 Creating a Partition

Syntax:

```
mkpart PARTITION-TYPE FILESYSTEM START END
```

Example:

```
mkpart primary ext4 1MiB 10GiB
```

Explanation:

Parameter	Meaning
-----------	---------

primary	Partition type
---------	----------------

ext4	Filesystem type
------	-----------------

1MiB	Start position
------	----------------

10GiB	End position
-------	--------------

5.2.7 Resizing Partition

Example:

```
resizepart 1 20GB
```

- **1** = partition number
 - **20GB** = new end position
-

5.2.8 Deleting a Partition

`rm 1`

- Deletes partition number 1
-

5.2.9 Viewing Partition Table

`print`

Example output:

Model: ATA VBOX HARDDISK

Disk /dev/sdb: 100GB

Sector size: 512B

Partition Table: gpt

Number	Start	End	Size	File system	Name	Flags
--------	-------	-----	------	-------------	------	-------

1	1049kB	10.7GB	10.7GB	ext4		
---	--------	--------	--------	------	--	--

5.2.10 Advantages of parted

- Works with **GPT and MBR**.
 - Supports **large disks >2TB**.
 - Can **resize partitions without data loss**.
 - Supports **scripted commands** for automation.
-

5.2.11 Disadvantages of parted

- CLI may be confusing for beginners.
 - Complex for simple partitioning compared to fdisk.
 - Not available in all minimal Linux installations (though most modern distributions have it).
-

5.3 Comparison Table: fdisk vs parted

Feature	fdisk	parted
Disk Type Supported	MBR (partially GPT)	MBR & GPT
Disk Size Supported	≤ 2TB	Any size
Resizing Partitions	No	Yes
Interface	Command-line interactive	Command-line interactive
Beginner Friendly	Moderate	Moderate to Advanced
Modern Usage	Older systems, small disks	Modern servers, cloud, large disks
Scripting Support	Limited	Yes

6. Filesystem Types in Linux

6.1 What is a Filesystem?

A **filesystem** is a method and data structure that an operating system uses to store, organize, and manage files on a disk or partition.

Without a filesystem:

- The OS cannot know **where a file starts or ends**.
 - You cannot **store or retrieve files efficiently**.
 - Disk space is just a “raw storage” without organization.
-

6.2 Why Filesystems Are Important

1. **Organizes Data** – Stores files in directories for easy access.
 2. **Manages Disk Space** – Divides disk into blocks, tracks free and used space.
 3. **Supports Metadata** – File ownership, permissions, timestamps, and access rights.
 4. **Ensures Reliability** – Handles power failures or crashes (journaling).
 5. **Optimizes Performance** – Fast file read/write through allocation strategies.
-

6.3 How Filesystems Work

A filesystem has several components:

Component	Purpose
-----------	---------

Superblock	Stores metadata about the filesystem: size, type, block size, free space, and inode count.
Inodes	Data structures storing file metadata (owner, permissions, timestamps, pointers to data blocks).
Data Blocks	Actual storage of file content.
Directory Structure	Maps filenames to inode numbers.
Journaling (Optional)	Logs changes to prevent corruption in case of crashes.

6.4 Types of Filesystems in Linux

Linux supports **many filesystem types**, but the most common ones are:

Filesystem	Description	Features	Typical Use
ext4	Fourth extended filesystem	Journaling, supports large files (16TB), stable and default in most distros	Root (/), Home (/home), general-purpose
ext3	Third extended filesystem	Journaling, smaller max file size (2TB), backward-compatible with ext2	Older systems, compatible upgrade to ext4

ext2	Second extended filesystem	Non-journaling, simple, reliable	Small devices, USB drives, SD cards
xfs	High-performance filesystem	Journaling, scalable, supports very large filesystems (8EB)	Large data storage, databases, servers
btrfs	Next-gen filesystem	Snapshots, compression, checksums for data integrity	Backup servers, modern systems, advanced storage
vfat / FAT32	Legacy DOS/Windows filesystem	Cross-platform compatibility	USB drives, SD cards for sharing with Windows/Mac
ntfs	Windows NT filesystem	Journaling, security features	Dual-boot with Windows, external drives
swap	Special filesystem	Used for virtual memory	Swap partitions for Linux

6.5 Key Linux Filesystems Explained

6.5.1. ext2

- **Non-journaling**, simplest Linux filesystem.
 - No log, so **faster for small disks**, but riskier in crashes.
 - Used on **USB drives** or older systems.
-

6.5.2. ext3

- Adds **journaling** to ext2 → safer, faster recovery after crash.
 - Journaling modes:
 - **writeback** – fast, but less safe
 - **ordered** – default, safer
 - **journal** – safest, slowest
 - Compatible with ext2 (can mount ext2 as ext3 without formatting).
-

6.5.3. ext4 (Most Common)

- Default in **Ubuntu, RHEL, CentOS, Fedora**.
 - Improvements over ext3:
 - Supports **huge files (16TB)**
 - Supports **huge filesystem size (1EB)**
 - Extents for efficient storage (less fragmentation)
 - Delayed allocation for better performance
 - Good for **root, home, data partitions**.
-

6.5.4. xfs

- Designed for **high-performance and large filesystems**.
- Features:
 - Journaling

- Parallel I/O (good for servers)
 - Very large file support (8 exabytes)
 - Commonly used in **enterprise servers, databases, virtualization storage**.
-

6.5.5. btrfs

- Modern filesystem with **snapshot and RAID support**.
 - Features:
 - Snapshots and rollback
 - Compression
 - Checksumming of data & metadata
 - Integrated multi-device support (RAID0/1/10)
 - Good for **backups, file servers, modern storage arrays**.
-

6.5.6. vfat / FAT32

- Cross-platform (Linux, Windows, Mac)
 - Limitations:
 - Max file size: 4GB
 - No Unix permissions
 - Use: **USB drives, SD cards, dual-boot devices**
-

6.5.7. NTFS

- Windows-native, supported by Linux via **ntfs-3g driver**.
 - Journaling and permissions supported.
 - Use: **External drives shared with Windows, dual-boot systems.**
-

6.5.8. swap

- Special “filesystem” for **virtual memory**.
 - Linux uses it when **RAM is full**.
 - Can be:
 - **Swap partition** → dedicated disk space
 - **Swap file** → file in ext4 filesystem
-

6.6 Choosing the Right Filesystem

Use Case	Recommended Filesystem
Linux root (/)	ext4
Home (/home)	ext4, xfs
Large database storage	xfs, btrfs
USB drives / SD cards	vfat / FAT32

Sharing with Windows NTFS

Backup & snapshots btrfs

Low-resource or small devices ext2

6.7 Creating Filesystems in Linux

After partitioning, **create a filesystem** using **mkfs**:

Create ext4 filesystem

```
sudo mkfs.ext4 /dev/sdb1
```

Create xfs filesystem

```
sudo mkfs.xfs /dev/sdb2
```

Create swap partition

```
sudo mkswap /dev/sdb3
```

```
sudo swapon /dev/sdb3
```

6.8 Checking Filesystem Type

View all filesystems

```
lsblk -f
```

Example output

NAME	FSTYPE	LABEL	UUID	MOUNTPOINT
sda1	ext4		12345678-90ab-cdef-1234-567890abcdef	/
sda2	xfs		98765432-10fe-dcba-9876-543210fedcba	/data
sdb1	vfat		11223344-5566-7788-99aa-bbccddeeff00	/mnt/usb

6.9 Mounting Filesystems

Mounting attaches a filesystem to the Linux directory tree.

Example:

```
sudo mount /dev/sdb1 /mnt/data
```

Now the disk is accessible at:

/mnt/data

6.10 View mounted filesystems

mount

or

lsblk

or

df -h

6.11 Unmounting Filesystems

Detach filesystem:

```
sudo umount /mnt/data
```


or

```
sudo umount /dev/sdb1
```

Important:

Filesystem cannot be unmounted if **in use**.

Check processes using disk:

```
ls -l /mnt/data
```

or

```
fuser -m /mnt/data
```

6.12 Persistent Mounting (fstab)

To mount disks automatically on boot, edit:

```
/etc/fstab
```

Example entry:

```
UUID=1234-5678 /data ext4 defaults 0 2
```

Fields explanation:

Field	Meaning
UUID	Disk identifier
Mount point	Directory
Filesystem	ext4, xfs

Options mount options

dump backup flag

fsck filesystem check order

Example:

```
/dev/sdb1 /data ext4 defaults 0 2
```

Apply changes:

```
sudo mount -a
```

6.13 Filesystem Check (fsck)

Used to check and repair filesystems.

Example:

```
sudo fsck /dev/sdb1
```

Important:

Filesystem should be **unmounted** before checking.

Example for ext4:

```
sudo fsck.ext4 /dev/sdb1
```



Complete Workflow with **fdisk**

1 Partitioning

Using **fdisk**

List current partitions

```
sudo fdisk -l /dev/sdb
```

Start fdisk

```
sudo fdisk /dev/sdb
```

Inside fdisk interactive mode:

1. Create primary partition

n → p → 1 → [Enter start] → +1G # /dev/sdb1

2. Create extended partition

n → e → 2 → [Enter start] → [Enter end] # /dev/sdb2

3. Create logical partition inside extended

n → l → [Enter start] → +1G # /dev/sdb5

4. Write changes

w

Verify:

```
sudo fdisk -l /dev/sdb
```

Should show:

/dev/sdb1 primary

/dev/sdb2 extended

/dev/sdb5 logical

2 Formatting Partitions

```
sudo mkfs.ext4 /dev/sdb1 # Primary
```

```
sudo mkfs.ext4 /dev/sdb5 # Logical
```

⚠ Extended partition itself **cannot be formatted**, only logical partitions inside it.

3 Create Mount Points

```
sudo mkdir -p /mnt/primary
```

```
sudo mkdir -p /mnt/logical
```

4 Mount Partitions Temporarily

```
sudo mount /dev/sdb1 /mnt/primary
```

```
sudo mount /dev/sdb5 /mnt/logical
```

- Verify mounts:

```
df -h | grep sdb
```

5 Unmount Partitions (Optional)

```
sudo umount /mnt/primary
```

```
sudo umount /mnt/logical
```

6 Make Mounts Permanent (via **/etc/fstab**)

Step 1: Get UUIDs

```
sudo blkid /dev/sdb1
```

```
sudo blkid /dev/sdb5
```

Sample output:

```
/dev/sdb1: UUID="1111-aaaa" TYPE="ext4"
```

```
/dev/sdb5: UUID="5555-bbbb" TYPE="ext4"
```

Step 2: Edit **/etc/fstab**

```
sudo nano /etc/fstab
```

Add:

```
UUID=1111-aaaa /mnt/primary ext4 defaults 0 2
```

```
UUID=5555-bbbb /mnt/logical /mnt/logical ext4 defaults 0 2
```

Step 3: Test

```
sudo mount -a
```

```
df -h | grep sdb
```

 Partitions are now **permanently mounted**.

Notes / Key Points

Concept	Notes
Primary Partition	Can have max 4 per MBR disk. Can be formatted and mounted directly.

Extended Partition	Only 1 allowed per MBR. Acts as a container for logical partitions. Cannot be formatted/mounted.
Logical Partition	Created inside extended partition. Numbering starts at 5. Can be formatted/mounted.
Formatting	Use <code>mkfs.ext4</code> (or other FS like xfs) to prepare partitions for storage.
Mounting	Temporary mount: <code>mount /dev/sdbX /mount/point</code> Unmount: <code>umount /mount/point</code>
Permanent Mount	Use <code>/etc/fstab</code> with UUID for reliability. Test with <code>mount -a</code> .
Partition Tools	<code>fdisk</code> → interactive MBR partitions <code>parted</code> → interactive or scripted, supports MBR/GPT, sizes in MiB/GiB
Verification	<code>fdisk -l /dev/sdb</code> → partitions <code>df -h</code> → mounted partitions <code>lsblk</code> → tree view of partitions and mount points



Complete Workflow with `parted`

1 Partitioning with `parted`

Start **parted**:

```
sudo parted /dev/sdb
```

Set disk label (MBR):

```
(parted) mklabel msdos
```

Create Primary Partition:

```
(parted) mkpart primary ext4 1MiB 1025MiB
```

- Creates `/dev/sdb1` (primary, 1 GiB).

Create Extended Partition:

```
(parted) mkpart extended 1025MiB 100%
```

- Creates `/dev/sdb2` (extended, rest of disk).

Create Logical Partition inside Extended:

```
(parted) mkpart logical ext4 1026MiB 2048MiB
```

- Creates `/dev/sdb5` (logical inside extended).

Verify partitions:

```
(parted) print
```

Expected output:

Number	Start	End	Size	Type
1	1MiB	1025MiB	1024MiB	primary
2	1025MiB	100%		extended
5	1026MiB	2048MiB		logical

Exit **parted**:

(parted) quit

2 Format Partitions

```
sudo mkfs.ext4 /dev/sdb1 # Primary
```

```
sudo mkfs.ext4 /dev/sdb5 # Logical
```

Extended partition `/dev/sdb2` cannot be formatted.

3 Create Mount Points

```
sudo mkdir -p /mnt/primary
```

```
sudo mkdir -p /mnt/logical
```

4 Mount Partitions Temporarily

```
sudo mount /dev/sdb1 /mnt/primary
```

```
sudo mount /dev/sdb5 /mnt/logical
```

- Verify:

```
df -h | grep sdb
```

5 Unmount Partitions (Optional)

```
sudo umount /mnt/primary
```

```
sudo umount /mnt/logical
```

6 Make Mounts Permanent (via **/etc/fstab**)

Step 1: Get UUIDs

```
sudo blkid /dev/sdb1
```

```
sudo blkid /dev/sdb5
```

Sample output:

```
/dev/sdb1: UUID="1111-aaaa" TYPE="ext4"
```

```
/dev/sdb5: UUID="5555-bbbb" TYPE="ext4"
```

Step 2: Edit **/etc/fstab**

```
sudo nano /etc/fstab
```

Add:

```
UUID=1111-aaaa /mnt/primary ext4 defaults 0 2
```

```
UUID=5555-bbbb /mnt/logical ext4 defaults 0 2
```

Step 3: Test **/etc/fstab**

```
sudo mount -a
```

```
df -h | grep sdb
```

 Partitions are now **permanently mounted**.



Notes / Key Points for **parted**

Concept

Notes

Primary Partition

First 1–4 partitions on MBR; can be mounted directly.

Extended Partition Container for logical partitions; cannot be mounted/formatted directly.

Logical Partition Inside extended; numbered from 5; can be mounted.

Disk Label `msdos` = MBR, `gpt` = GPT. Use `mklabel msdos` for MBR.

Formatting `mkfs.ext4 /dev/sdbX` prepares partitions.

Mounting Temporary: `mount /dev/sdbX /mount/point`.

Unmount: `umount /mount/point`.

Permanent Mount Use UUID in `/etc/fstab`. Test with `mount -a`.

Verification `parted /dev/sdb print` → partitions.

`df -h` → mounted partitions.

`lsblk` → tree view.



Complete GPT Partition Example Using `parted`

1 Start `parted`

`sudo parted /dev/sdc`

2 Set GPT Disk Label

```
(parted) mklabel gpt
```

GPT allows **up to 128 partitions by default** and doesn't use "extended/logical" concept like MBR. Every partition is essentially a primary partition.

3 Create Partitions

Partition 1 (Primary)

```
(parted) mkpart primary ext4 1MiB 1025MiB
```

- `/dev/sdc1` → 1 GiB

Partition 2

```
(parted) mkpart primary ext4 1025MiB 2049MiB
```

- `/dev/sdc2` → 1 GiB

Partition 3

```
(parted) mkpart primary ext4 2049MiB 3073MiB
```

- `/dev/sdc3` → 1 GiB

Verify Partitions

```
(parted) print
```

Expected output:

Number	Start	End	Size	File system	Name
1	1MiB	1025MiB	1024MiB	ext4	
2	1025MiB	2049MiB	1024MiB	ext4	

3 2049MiB 3073MiB 1024MiB ext4

- Exit parted:

(parted) quit

4 Format Partitions

```
sudo mkfs.ext4 /dev/sdc1
```

```
sudo mkfs.ext4 /dev/sdc2
```

```
sudo mkfs.ext4 /dev/sdc3
```

5 Create Mount Points

```
sudo mkdir -p /mnt/gpt1
```

```
sudo mkdir -p /mnt/gpt2
```

```
sudo mkdir -p /mnt/gpt3
```

6 Mount Partitions Temporarily

```
sudo mount /dev/sdc1 /mnt/gpt1
```

```
sudo mount /dev/sdc2 /mnt/gpt2
```

```
sudo mount /dev/sdc3 /mnt/gpt3
```

- Verify:

```
df -h | grep sdc
```

7 Make Mounts Permanent (via `/etc/fstab`)

Step 1: Get UUIDs

```
sudo blkid /dev/sdc1
```

```
sudo blkid /dev/sdc2
```

```
sudo blkid /dev/sdc3
```

Sample output:

```
/dev/sdc1: UUID="aaaa-1111" TYPE="ext4"
```

```
/dev/sdc2: UUID="bbbb-2222" TYPE="ext4"
```

```
/dev/sdc3: UUID="cccc-3333" TYPE="ext4"
```

Step 2: Edit `/etc/fstab`

```
sudo nano /etc/fstab
```

Add:

```
UUID=aaaa-1111 /mnt/gpt1 ext4 defaults 0 2
```

```
UUID=bbbb-2222 /mnt/gpt2 ext4 defaults 0 2
```

```
UUID=cccc-3333 /mnt/gpt3 ext4 defaults 0 2
```

Step 3: Test `/etc/fstab`

```
sudo mount -a
```

```
df -h | grep sdc
```

 Partitions are now **permanently mounted**.



Notes / Key Points for GPT

Concept	Notes
GPT vs MBR	GPT allows 128+ partitions, no need for extended/logical partitions. Each is “primary.”
Disk Label	<code>gpt</code> → use <code>parted mklabel gpt</code> .
Formatting	Same as MBR: <code>mkfs.ext4 /dev/sdcX</code> .
Mounting	Temporary: <code>mount /dev/sdcX /mount/point</code> Permanent: via <code>/etc/fstab</code> using UUIDs.
Verification	<code>parted /dev/sdc print</code> → partition table <code>lsblk</code> → tree view <code>df -h</code> → mounted partitions

The file `/etc/fstab` (File System Table) defines **how storage devices are automatically mounted when the system boots**.

Each line in `/etc/fstab` has **6 fields** separated by spaces or tabs.

7. Structure of `/etc/fstab`

General format:

`<device> <mount_point> <filesystem_type> <mount_options> <dump> <pass>`

Example entry:

```
UUID=3c2f4c2d-1c2f-4b3f-a9d0-12e8d0b4c3a0 /data ext4 defaults 0 2
```

7.1 Field 1 — Device (Filesystem Source)

This specifies **which device or filesystem should be mounted**.

Common ways to specify a device:

1 Device name

/dev/sdb1

Example:

```
/dev/sdb1 /data ext4 defaults 0 2
```

2 UUID (recommended)

```
UUID=3c2f4c2d-1c2f-4b3f-a9d0-12e8d0b4c3a0
```

Find UUID using:

blkid

UUID is preferred because **device names may change after reboot**.

3 LABEL

```
LABEL=mydisk
```

Set label using:

```
e2label /dev/sdb1 mydisk
```

7.2 Field 2 — Mount Point

This specifies **where the filesystem will be mounted in the directory tree**.

Examples:

/

/home

/data

Example entry:

```
/dev/sdb1 /data ext4 defaults 0 2
```

Mount point must exist before mounting.

Example:

```
mkdir /data
```

7.3 Field 3 — Filesystem Type

Specifies the **type of filesystem on the device**.

Common types:

Filesystem	Usage
ext4	common Linux filesystem
xfs	default in RHEL/CentOS
swap	swap partition
vfat	FAT filesystem
ntfs	Windows filesystem
iso9660	CD/DVD
nfs	network filesystem

Example:

```
ext4  
xfs
```

7.4 Field 4 — Mount Options

Defines **how the filesystem should be mounted**.

Options are separated by commas.

Common options

Option	Meaning
defaults	default mount options
ro	read only
rw	read/write
noexec	prevent executing binaries
nosuid	disable SUID programs
nodev	ignore device files
auto	mount automatically at boot
noauto	do not mount automatically

Example:

defaults

Example with multiple options:

rw,nosuid,nodev

The **4th field in /etc/fstab** is called **mount options**.

It tells Linux **how the filesystem should behave when it is mounted**.

Think of it like **rules for using the disk**.

Example **/etc/fstab** entry:

```
UUID=123-456 /data ext4 defaults 0 2
```

Here:

Field	Meaning
1	Device
2	Mount point

- 3 Filesystem type
- 4 Mount options
(rules)**
- 5 dump
- 6 fsck

So the **4th field controls how the disk is used.**

1. The Most Common Option: **defaults**

Example:

UUID=123 /data ext4 defaults 0 2

defaults means Linux automatically applies several normal options.

It is the same as writing:

rw,suid,dev,exec,auto,nouser,async

Let's explain these in **very simple terms.**

2. Common Mount Options (Layman Explanation)

1. **rw** – Read and Write

rw

Meaning:

You can **read files and also modify them.**

Example:

rw

You can:

- create files
- edit files
- delete files

Opposite option:

ro

Meaning:

Read only.

Example:

UUID=123 /data ext4 ro 0 2

Now users **cannot modify anything**.

Good for:

- backup disks
- recovery partitions

2. **exec** – Allow Programs to Run

exec

Meaning:

Programs stored on this disk **can run**.

Example:

/data/script.sh

If **exec** is enabled → script runs.

Opposite:

noexec

Programs **cannot run from that disk**.

Example use:

USB drives or shared folders for security.

3. **dev** – Allow Device Files

dev

Meaning:

Linux allows **device files** like:

/dev/sda

/dev/null

Opposite:

nodev

Meaning:

Device files **are ignored**.

This improves security for some partitions.

4. **suid** – Allow Special Privileges

suid

Some programs can run with **special permissions**.

Example:

passwd

Even normal users can change passwords because of **SUID bit**.

Opposite:

nosuid

Meaning:

Ignore special permissions.

Used for **security reasons**.

5. **auto** – Mount Automatically at Boot

auto

Meaning:

Linux mounts the filesystem **automatically during boot**.

Opposite:

noauto

Meaning:

Linux **will NOT mount automatically**.

You must mount it manually:

mount /data

6. **nouser** – Only Root Can Mount

nouser

Meaning:

Only **root user can mount the disk**.

Opposite:

user

Meaning:

Normal users **can mount the filesystem**.

Useful for USB drives.

7. **async** – Faster Disk Writing

async

Linux writes data **a little later**, improving speed.

Opposite:

sync

Meaning:

Write data **immediately**.

More safe but slower.

7.5 Field 5 — Dump

Used by the **dump backup utility**.

Values:

Value	Meaning
0	disable dump
1	enable dump backup

Most systems use:

0

Example:

/dev/sdb1 /data ext4 defaults 0 2

7.6 Field 6 — Pass (Filesystem Check Order)

Controls the order in which **fsck** checks filesystems during boot.

Values:

Value	Meaning
0	skip filesystem check
1	check first (root filesystem)
2	check after root

Typical configuration:

/ → pass = 1
other partitions → pass = 2

Example:

UUID=xxxx / ext4 defaults 0 1
UUID=yyyy /data ext4 defaults 0 2

7.7 Complete Example **/etc/fstab**

```
UUID=abc123 / xfs defaults 0 1
UUID=def456 /home xfs defaults 0 2
UUID=ghi789 /data ext4 defaults 0 2
UUID=jkl111 none swap sw 0 0
```

Explanation:

/ → root filesystem
/home → home directory
/data → additional storage
swap → swap space

7.8 Testing **/etc/fstab**

After editing **/etc/fstab**, always test with:

mount -a

If no errors appear, the entry is correct.

7.9 Important Rules

- ① Always use **UUID** instead of device name
 - ② Mount point directory must exist
 - ③ Incorrect entries can **prevent system boot**
-

8. Common Mistakes in `/etc/fstab`

The `/etc/fstab` file is very powerful, but even a small mistake can make the **system fail to boot** or prevent filesystems from mounting correctly. Below are **5 very common mistakes** people make in `/etc/fstab`, especially beginners learning Linux administration.

8.1 Using the Wrong Device Name Instead of UUID

Many people write entries like:

```
/dev/sdb1 /data ext4 defaults 0 0
```

8.1.1 Problem

Device names like `/dev/sdb1`, `/dev/sdc1` can **change after reboot** depending on hardware detection order.

Example:

- Today: `/dev/sdb1`
- After reboot: becomes `/dev/sdc1`

Result: **Filesystem will not mount.**

8.1.2 Correct Practice

Use **UUID** instead.

Find UUID:

blkid

Example entry:

```
UUID=3e6be9de-8139-11d1-9106-a43f08d823a6 /data ext4 defaults 0 0
```

UUID **never changes**, so it's safer.

8.2 Mount Point Directory Does Not Exist

Example entry:

```
UUID=abc123 /backup ext4 defaults 0 0
```

8.2.1 Problem

If **/backup** directory does not exist, the system **cannot mount the filesystem**.

8.2.2 Error

mount: mount point does not exist

8.2.3 Fix

Create mount point first:

```
mkdir /backup
```

Then mount.

8.3 Wrong Filesystem Type

Example:

```
UUID=abc123 /data xfs defaults 0 0
```

But the actual filesystem is **ext4**.

8.3.1 Result

Mount fails with error:

wrong fs type, bad option, bad superblock

8.3.2 Check filesystem type

lsblk -f

or

blkid

Correct entry:

UUID=abc123 /data ext4 defaults 0 0

8.4 Wrong Dump or FSCK Values (Last Two Fields)

Example mistake:

UUID=abc123 /data ext4 defaults 1 1

8.4.1 Problem

1 in the **last column** tells system to check filesystem at boot.

But **only root filesystem should be 1**.

Correct order:

Filesystem	FSCK Value
------------	------------

/ root	1
--------	---

other partitions	2
------------------	---

no check 0

8.4.2 Correct Example

```
UUID=rootuuid / ext4 defaults 1 1
```

```
UUID=atauid /data ext4 defaults 0 2
```

8.5 Not Testing **/etc/fstab** After Editing

This is the **most dangerous mistake**.

If you edit **/etc/fstab** and reboot without testing, system may drop into **emergency mode**.

8.5.1 Always Test Using

```
mount -a
```

What it does:

- Tries to mount **everything from /etc/fstab**
- Shows errors without rebooting

Example workflow:

```
vi /etc/fstab
```

```
mount -a
```

If no error → safe to reboot.

8.6 Bonus Mistake (Very Common)

8.6.1 Spaces in Mount Options

Wrong:

```
UUID=abc123 /data ext4 defaults, noatime 0 0
```

Space after comma causes parsing issue.

Correct:

```
UUID=abc123 /data ext4 defaults,noatime 0 0
```

8.7 Pro Admin Tip (Very Important)

When experimenting with `/etc/fstab`, add **nofail** option.

Example:

```
UUID=abc123 /data ext4 defaults,nofail 0 2
```

Meaning:

- If mount fails → **system still boots**

Without **nofail**, boot may **stop in emergency mode**.

8.8 Safe Workflow Used by Linux Admins

1. Add entry to `/etc/fstab`
2. Create mount directory
3. Test with:

```
mount -a
```

4. Verify:

```
df -h
```

5. Then reboot.

9. Linux Filesystem Resizing Notes (EXT & XFS using fdisk) using fdisk

These notes explain **how to grow and shrink EXT and XFS filesystems using fdisk** with clear explanations, commands, and examples used by Linux administrators.

9.1 Important Concepts

Before resizing any filesystem, understand these key rules.

Filesystem vs Partition

- **Partition** → Logical section of a disk (created using fdisk/parted).
- **Filesystem** → Structure inside a partition that stores files (ext4, xfs, etc.).

Example

Disk

/dev/sdb

Partition

/dev/sdb1

Filesystem

ext4 or xfs

Mounted as

/data

9.2 EXT vs XFS Resizing Capabilities

Feature	EXT (ext2/ext3/ext4)	XFS
Grow filesystem	Yes	Yes
Shrink filesystem	Yes	No
Online resize	Limited	Yes
Offline resize	Yes	Usually not needed

Key rule:

EXT

- Can grow
- Can shrink

XFS

- Can grow
 - Cannot shrink
-

9.3 Important Safety Rules

Before resizing:

1. Always **take backup**.
2. Always **check filesystem using fsck** before shrinking EXT.
3. When recreating partition using fdisk:
 - **Start sector must be the same.**
4. Shrinking order:

Filesystem shrink → Partition shrink

Growing order:

Partition grow → Filesystem grow

9.4 Expanding EXT Filesystem using fdisk

Example Scenario

Current disk

/dev/sdb1 ext4 10G

Disk expanded to

20G

Goal

Increase filesystem to 20G

Step 1: Check disk layout

lsblk

df -h

Example

```
sdb    20G
```

```
└─sdb1 10G
```

Step 2: Unmount filesystem

```
umount /dev/sdb1
```

Step 3: Check filesystem

```
fsck -f /dev/sdb1
```

Step 4: Open fdisk

```
fdisk /dev/sdb
```

Print partitions

```
p
```

Example

Device	Start	End	Size
/dev/sdb1	2048	20973567	10G

Note **Start sector (2048)**.

Step 5: Delete partition entry

d

1

This deletes **only partition table entry**, not data.

Step 6: Recreate partition

n

p

1

Enter **same start sector**

2048

For last sector press Enter to use full disk.

Step 7: Save changes

w

Reload partition table

partprobe

Step 8: Resize EXT filesystem

`resize2fs /dev/sdb1`

Verify

`df -h`

Result

`/dev/sdb1 20G`

9.5 Expanding XFS Filesystem using fdisk

Important rule:

XFS must be mounted to grow.

Example

Current

`/dev/sdb1 xfs 10G`

Disk expanded to

20G

Step 1: Extend partition using fdisk

`fdisk /dev/sdb`

Inside fdisk

`p`

`d`

`n`

Recreate partition with **same start sector**.

Write changes

`w`

Reload table

`partprobe`

Step 2: Grow XFS filesystem

If mounted on `/data`

`xfs_growfs /data`

or

`xfs_growfs /dev/sdb1`

Verify

`df -h`

9.6 Shrinking EXT Filesystem using fdisk

EXT supports shrinking but must be done carefully.

Example Scenario

Current

`/dev/sdb1 ext4 10G`

Target

5G

Step 1: Check disk usage

`df -h /data`

Example

Filesystem	Size	Used
------------	------	------

<code>/dev/sdb1</code>	10G	2G
------------------------	-----	----

Used space must be **less than new size**.

Step 2: Unmount filesystem

```
umount /dev/sdb1
```

Step 3: Check filesystem

```
fsck -f /dev/sdb1
```

Step 4: Shrink filesystem

```
resize2fs /dev/sdb1 5G
```

Now filesystem is reduced.

Step 5: Modify partition using fdisk

```
fdisk /dev/sdb
```

Print partitions

```
p
```

Example

Device	Start	End
/dev/sdb1	2048	20973567

Note **Start sector (2048)**.

Step 6: Delete partition entry

d

1

Step 7: Recreate smaller partition

n

p

1

Start sector

2048

End size

+5G

Step 8: Save changes

w

Reload table

partprobe

Step 9: Mount filesystem

```
mount /dev/sdb1 /data
```

Verify

```
df -h
```

Result

```
/dev/sdb1 5G
```

9.7 Shrinking XFS Filesystem

XFS cannot be shrunk.

Linux does not support shrinking XFS.

Correct method:

1. Backup data
 2. Delete partition
 3. Create smaller partition
 4. Create new XFS filesystem
 5. Restore data
-

Example

Current

```
/dev/sdb1 xfs 10G
```

Target

5G

Step 1: Backup data

```
rsync -av /data /backup
```

or

```
cp -a /data /backup
```

Step 2: Unmount filesystem

```
umount /data
```

Step 3: Delete partition

```
fdisk /dev/sdb
```

d

1

Step 4: Create smaller partition

n

p

1

Start sector

2048

Size

+5G

Save

w

Step 5: Create new filesystem

mkfs.xfs /dev/sdb1

Step 6: Mount filesystem

mount /dev/sdb1 /data

Step 7: Restore data

```
rsync -av /backup/data/ /data/
```

10. Linux Filesystem Resizing using parted (EXT & XFS)

10.1 Check Disk Layout

Before resizing any partition.

```
lsblk  
df -h
```

Example

```
NAME  SIZE TYPE MOUNTPOINT  
sdb   20G disk  
└─sdb1 10G part /data
```

Check partition details:

```
parted /dev/sdb print
```

Example output:

Number	Start	End	Size	Type
1	1MB	10GB	10GB	primary

10.2 Growing EXT Filesystem using parted

Example Scenario

Current:

```
/dev/sdb1 ext4 10G
```

Disk increased to:

20G

Goal:

Expand partition to 20G

Step 1: Open parted

parted /dev/sdb

Step 2: View partitions

print

Example:

Number	Start	End	Size
1	1MB	10GB	10GB

Step 3: Resize partition

resizepart 1 20GB

This expands partition **1** to **20GB**.

Confirm:

Yes/No → Yes

Step 4: Exit parted

quit

Step 5: Resize EXT filesystem

Now grow filesystem.

resize2fs /dev/sdb1

Step 6: Verify

`df -h`

Output

`/dev/sdb1 20G`

10.3 Growing XFS Filesystem using **parted**

Important rule:

XFS must be **mounted to grow**.

Example

`/dev/sdb1 xfs 10G`

Target

`20G`

Step 1: Resize partition

`parted /dev/sdb`

Inside parted:

`print
resizepart 1 20GB
quit`

Step 2: Grow filesystem

If mounted at `/data`

`xfs_growfs /data`

or

`xfs_growfs /dev/sdb1`

Step 3: Verify

`df -h`

Result

`/dev/sdb1 20G`

10.4 Shrinking EXT Filesystem using **parted**

Shrinking EXT requires **filesystem shrink first**.

Example

Current

`/dev/sdb1 ext4 10G`

Target

5G

Step 1: Check usage

`df -h /data`

Ensure data < new size.

Example

Used 2G

Step 2: Unmount filesystem

`umount /data`

Step 3: Check filesystem

`fsck -f /dev/sdb1`

Step 4: Shrink filesystem

```
resize2fs /dev/sdb1 5G
```

Now filesystem is reduced.

Step 5: Resize partition using parted

```
parted /dev/sdb
```

Inside parted:

```
print
resizepart 1 5GB
quit
```

Step 6: Mount filesystem

```
mount /dev/sdb1 /data
```

Step 7: Verify

```
df -h
```

Output

```
/dev/sdb1 5G
```

10.5 Shrinking XFS Filesystem

✗ XFS cannot shrink.

Linux does **not support shrinking XFS**.

Only method:

- ① Backup data
- ② Delete partition
- ③ Create smaller partition
- ④ Create new XFS filesystem
- ⑤ Restore data

Example Process

Step 1: Backup

```
rsync -av /data /backup
```

Step 2: Unmount

```
umount /data
```

Step 3: Delete partition

```
parted /dev/sdb
```

Inside parted:

```
rm 1
```

Step 4: Create smaller partition

```
mkpart primary xfs 1MB 5GB
```

Step 5: Create filesystem

```
mkfs.xfs /dev/sdb1
```

Step 6: Mount

```
mount /dev/sdb1 /data
```

Step 7: Restore

```
rsync -av /backup/data/ /data/
```

10.6 Important Linux Admin Rules

10.6.1 Shrinking order

Filesystem shrink

↓

Partition shrink

10.6.2 Growing order

Partition grow

↓

Filesystem grow

10.7 Parted Units Cheat Sheet

Unit	Size in Bytes	Notes / Base
1KiB	1024 B	Binary (2^{10}), i.e., powers of 2
1MiB	1,048,576 B	Binary (2^{20})
1GiB	1,073,741,824 B	Binary (2^{30})
1KB	1,000 B	Decimal (10^3), SI standard
1MB	1,000,000 B	Decimal (10^6)
1GB	1,000,000,000 B	Decimal (10^9)

10.8 Alternative: use MiB explicitly

If you prefer, you can also specify size in **MiB**:

(parted) unit MiB

(parted) resizepart 1 5120MiB

- 1 GB = 1024 MiB
- So 5 GB = 5120 MiB

This avoids all unit-related errors.

11. Linux Logical Volume Manager (LVM)

1 What is LVM?

LVM (Logical Volume Manager) is a storage management system in Linux that allows flexible disk management compared to traditional disk partitioning.

It abstracts physical storage devices and allows you to create **logical storage volumes** that can be resized easily.

Instead of using fixed partitions like `/dev/sda1`, `/dev/sda2`, LVM allows you to combine multiple disks and create flexible volumes.

Key Advantages

- Resize storage **without rebooting**
 - Combine **multiple disks into one pool**
 - Create **snapshots for backups**
 - Easily **extend or reduce storage**
 - Move data between disks without downtime
-

2 Problems with Traditional Partitioning

Traditional disk partitions are **static and rigid**.

Example:

Disk: **100GB**

Partition	Size	Mount
<code>/dev/sda1</code>	30GB	<code>/</code>
<code>/dev/sda2</code>	20GB	<code>/home</code>

/dev/sda3 50GB /var

2.1 Problem Scenario

If /home becomes full:

- /home = 20GB full
- /var = 50GB but only using 10GB

You **cannot easily move space** from /var to /home.

Possible solutions (traditional):

- Backup data
- Delete partitions
- Recreate partitions
- Restore backup

This is **risky and time-consuming**.

2.2 How LVM Solves This Problem

LVM creates a **storage abstraction layer**.

Instead of directly using partitions:

Disk → Physical Volume → Volume Group → Logical Volume → Filesystem

Example:

/dev/sda

/dev/sdb

↓

Physical Volumes

↓

Volume Group (VG)

↓
Logical Volumes (LV)
↓
Filesystem (/home /var /data)

Now storage becomes **dynamic**.

You can:

- Increase `/home` anytime
- Add new disks to VG
- Move volumes across disks

2.3 Traditional disk partitioning has rigid limitations:

Limitation	Explanation	Example
Fixed size partitions	Once you create a partition, resizing is risky.	<code>/dev/sda1 = 10G, /dev/sda2 = 5G. /dev/sda1 runs out of space → difficult to expand.</code>
Cannot combine disks easily	Each partition exists on a single disk.	Need 50G extra storage → must add a new disk and mount separately.
Difficult online resizing	Usually requires unmounting and sometimes reboot.	Cannot easily extend <code>/var</code> while system is running.
Snapshots/backups	No built-in snapshot feature.	Take a consistent snapshot of <code>/home</code> → manual and risky.
Fragmented storage management	Space can be wasted if partitions are partially used.	<code>/var</code> 50G mostly empty, <code>/home</code> 80% full → cannot easily balance.

3 Why LVM is Needed

LVM solves the above problems by:

1. Dynamic resizing of storage

- Add or reduce storage **without rebooting** (EXT can shrink, EXT/XFS can grow).

2. Combining multiple physical disks

- Create **one large logical volume** from multiple disks.

3. Snapshots

- Create **point-in-time snapshots** for backup or testing.

4. Flexible storage allocation

- Logical volumes can be resized as per application needs.

5. Improved storage utilization

- Avoid wasted space on underutilized partitions.
-

4 Why Do We Need LVM if Standard Partitions Can Be Resized?

Yes, **standard Linux partitions (fdisk/parted)** can be resized, but they have **major limitations**. LVM solves many real-world problems administrators face.

1. Flexibility (Resize Without Disk Limitations)

Standard Partition

- Must resize **the actual partition**
- Requires **free space next to the partition**
- Often needs **unmounting or downtime**

Example:

```
/dev/sda1 20GB /  
/dev/sda2 20GB /home
```

Free space 10GB

If / becomes full but free space is **after** /home, resizing / is difficult.

LVM

You can simply extend the logical volume.

Example:

VG: vg_data (50GB total)

LV_root 20GB /

LV_home 20GB /home

Free 10GB

If / becomes full:

```
lvextend -L +5G /dev/vg_data/lv_root
```

```
resize2fs /dev/vg_data/lv_root
```

No partition movement required.

2. Combine Multiple Disks into One Volume

Standard partitions **cannot merge disks easily**.

Example:

Disk1 = 100GB

Disk2 = 100GB

With LVM:

PV1 (/dev/sda)

PV2 (/dev/sdb)

VG = 200GB pool

Now create:

LV_data = 150GB

Even though no single disk is 150GB.

3. Online Resizing (Very Important in Production)

Many LVM filesystems allow **online expansion**.

Example:

```
lvextend -L +10G /dev/vg_data/lv_home  
resize2fs /dev/vg_data/lv_home
```

System **continues running without reboot**.

4. Snapshots (Backup Feature)

LVM allows **snapshots**, useful for backups.

Example:

```
lvcreate --snapshot -L 2G -n data_snap /dev/vg_data/lv_data
```

You can back up data from snapshot **without affecting the running system**.

5. Easier Disk Expansion in Servers

Real scenario:

A server disk is full:

/dev/sda = 100GB

You add another disk:

/dev/sdb = 100GB

With LVM:

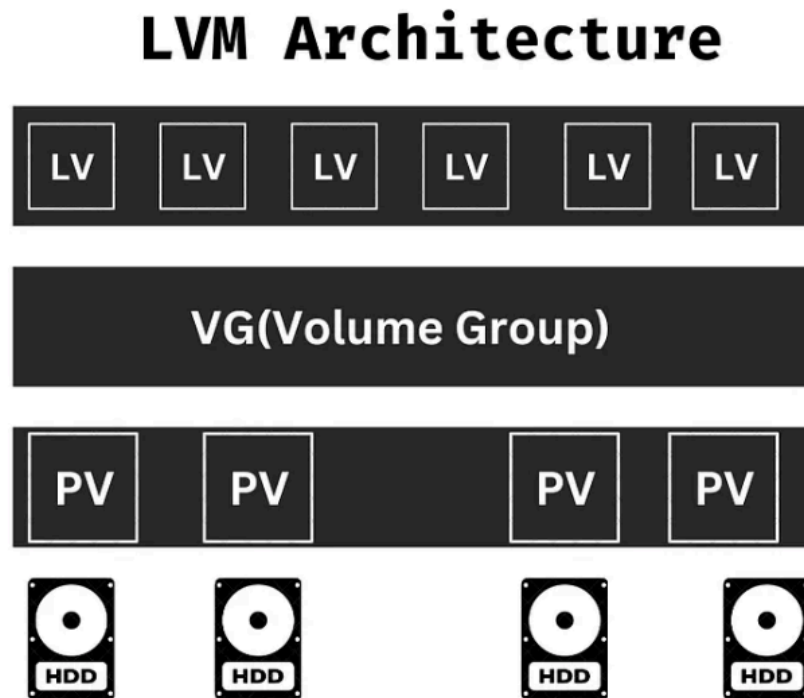
```
pvccreate /dev/sdb  
vgextend vg_data /dev/sdb  
lvextend -L +50G /dev/vg_data/lv_root
```

Now root filesystem grows **without repartitioning**.

5 LVM Architecture & Terminology

LVM introduces a **3-layer structure**:

PV=Physical volume
VG=Volume Group
LV=Logical Volume



1. Physical Volume (PV)

A **Physical Volume** is a disk or partition used by LVM.

- Physical storage: partition, disk, RAID.
- Command to create:

Examples:

/dev/sdb
/dev/sdc
/dev/sda3

These disks are initialized using:

`pvccreate`

Example:

`pvccreate /dev/sdb`

Check PV:

`pvddisplay`

`pvs`

2. Volume Group (VG)

A **Volume Group** is a **pool of storage created by combining multiple PVs**.

- Pool of storage created from one or more PVs.
- Acts as **virtual storage container**.
- Space in VG divided into **Physical Extents (PE)**.

Example:

PV1 = /dev/sdb = 50GB

PV2 = /dev/sdc = 50GB

VG = vg_data = 100GB

Command to create VG:

`vgcreate vg_data /dev/sdb /dev/sdc`

Check VG:

`vgdisplay`

`vgs`

3. Logical Volume (LV)

Logical Volumes are like **virtual partitions** created from a Volume Group.

- “Virtual partition” inside a VG.
- Filesystem created on LV.
- Example:

```
lvcreate -L 50G -n lv_home vg_data
mkfs.ext4 /dev/vg_data/lv_home
mount /dev/vg_data/lv_home /home
```

- LV space divided into **Logical Extents (LE)** that map to PE in VG.

Example:

VG = 100GB

Logical Volume	Size	Mount
lv_root	30GB	/
lv_home	40GB	/home
lv_backup	30GB	/backup

Create LV:

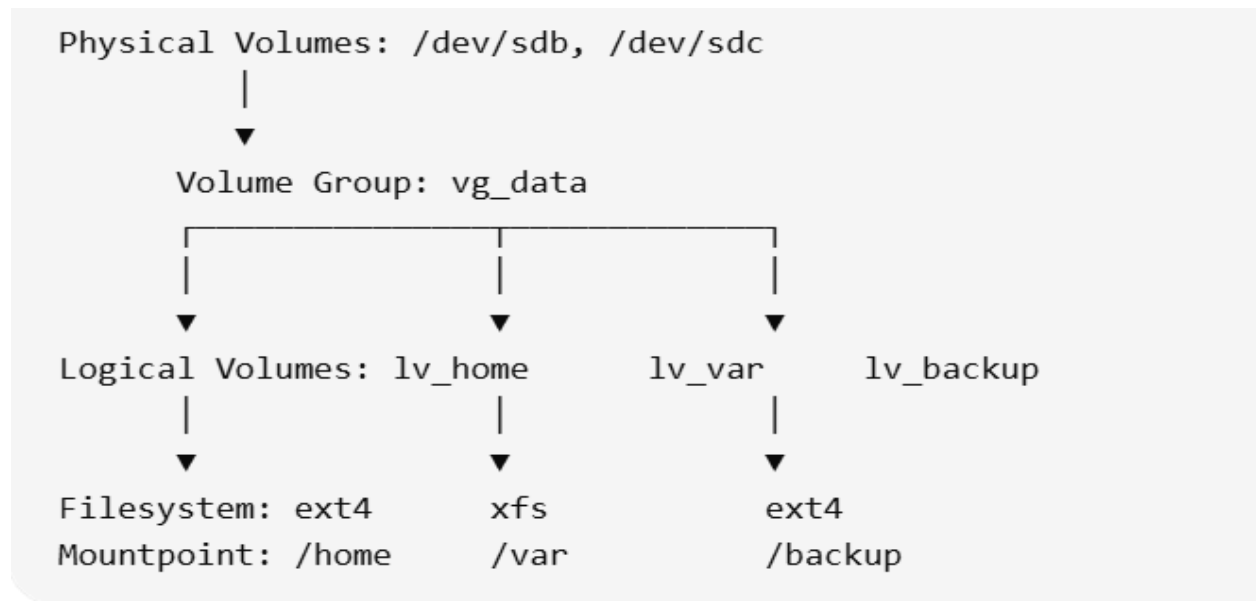
```
lvcreate -L 20G -n lv_data vg_data
```

Check LV:

```
lvdisplay
```

```
lvs
```

4. Diagram of LVM Structure



6 Advantages of LVM over Traditional Partitioning

1. Resize volumes **without rebooting**
 2. Combine **multiple disks into a single pool**
 3. **Snapshots** for backup or testing
 4. **Move LVs between PVs**
 5. Better **storage utilization**
 6. Manage **large disks (>2TB)** easily
-

7 When to Use LVM

- Servers that require **dynamic storage allocation**
- Virtual machines and cloud servers

- Databases where storage usage grows over time
 - Systems where **downtime must be minimized**
 - Systems needing **snapshots** for backup/testing
 - Expecting **future disk expansion**
 - Need **flexible storage**
 - Managing **multiple disks**
-

8 When Standard Partitions Are Enough

Standard partitions are fine for:

- Small systems
 - Personal laptops
 - Systems where storage rarely changes
-

9 LVM Commands Cheat Sheet

Task	Command
Create PV	<code>pvcreate /dev/sdb</code>
List PVs	<code>pvs</code> or <code>pvdisk</code>
Create VG	<code>vgcreate vg_data /dev/sdb /dev/sdc</code>
List VGs	<code>vgs</code> or <code>vgdisplay</code>
Create LV	<code>lvcreate -L 50G -n lv_home vg_data</code>
List LVs	<code>lvs</code> or <code>lvdisplay</code>
Remove LV	<code>lvremove /dev/vg_data/lv_home</code>

Grow LV	<code>lvextend -L +10G /dev/vg_data/lv_home</code>
Shrink LV	<code>lvreduce -L 40G /dev/vg_data/lv_home</code>
Grow EXT filesystem	<code>resize2fs /dev/vg_data/lv_home</code>
Grow XFS filesystem	<code>xfs_growfs /mountpoint</code>

12. Linux LVM Management: Create, Resize, Delete

1 Creating LVM

LVM is made of **Physical Volumes (PV)**, **Volume Groups (VG)**, and **Logical Volumes (LV)**.

Step 1: Create Physical Volumes (PV)

PV is the **physical storage** (disk/partition) used by LVM.

```
# Example: create PV on /dev/sdb and /dev/sdc  
pvcreate /dev/sdb /dev/sdc
```

Verify:

```
pvs
```

Output example:

PV	VG	Fmt	Attr	PSize	PFree
/dev/sdb		lvm2	---	50.00G	50.00G
/dev/sdc		lvm2	---	50.00G	50.00G

Step 2: Create Volume Group (VG)

VG is a **pool of storage** made from one or more PVs.

```
# Create VG named vg_data
vgcreate vg_data /dev/sdb /dev/sdc
```

Verify:

```
vgs
vgdisplay vg_data
```

Step 3: Create Logical Volume (LV)

LV acts like a **partition** inside the VG.

```
# Create 50G LV named lv_home in vg_data
lvcreate -L 50G -n lv_home vg_data
```

Verify:

```
lvs
lvdisplay /dev/vg_data/lv_home
```

Step 4: Create Filesystem on LV

```
mkfs.ext4 /dev/vg_data/lv_home
```

Step 5: Mount LV

```
mkdir /home
mount /dev/vg_data/lv_home /home
df -h /home
```

Resizing LVM

LVM makes it easy to **grow or shrink** logical volumes.

A. Growing LVM (EXT/XFS)

1. Grow LV

```
# Add 10G to lv_home
```

```
lvextend -L +10G /dev/vg_data/lv_home
```

2. Grow Filesystem

- For **EXT4**:

```
resize2fs /dev/vg_data/lv_home
```

- For **XFS**:

```
xfs_growfs /home
```

3. Verify

```
df -h /home
```

✅ Now the LV has increased space.

B. Reducing LVM (Only EXT)

⚠️ **XFS cannot be reduced.** Only EXT filesystems can shrink safely.

1. Unmount LV

```
umount /home
```

2. Check filesystem

```
fsck -f /dev/vg_data/lv_home
```

3. Shrink filesystem first

```
resize2fs /dev/vg_data/lv_home 40G
```

4. Reduce LV size

```
lvreduce -L 40G /dev/vg_data/lv_home
```

5. Mount LV

```
mount /home  
df -h /home
```

3 Deleting LVM

To completely remove LVM:

Step 1: Unmount LV

```
umount /home
```

Step 2: Remove LV

```
lvremove /dev/vg_data/lv_home
```

Confirm with **y**.

Step 3: Remove VG

```
vgremove vg_data
```

Step 4: Remove PV

```
pvremove /dev/sdb /dev/sdc
```

4 Summary of LVM Commands

Task	Command
Create PV	<code>pvcreate /dev/sdb</code>
Display PV	<code>pvs</code> or <code>pvdisk</code>
Create VG	<code>vgcreate vg_data /dev/sdb</code>

Display VG	<code>vgs</code> or <code>vgdisplay vg_data</code>
Create LV	<code>lvcreate -L 50G -n lv_home vg_data</code>
Display LV	<code>lvs</code> or <code>lvdisplay /dev/vg_data/lv_home</code>
Grow LV	<code>lvextend -L +10G /dev/vg_data/lv_home</code>
Shrink LV	<code>lvreduce -L 40G /dev/vg_data/lv_home</code>
Resize EXT FS	<code>resize2fs /dev/vg_data/lv_home</code>
Grow XFS FS	<code>xfs_growfs /mountpoint</code>
Remove LV	<code>lvremove /dev/vg_data/lv_home</code>
Remove VG	<code>vgremove vg_data</code>
Remove PV	<code>pvremove /dev/sdb</code>

13. Physical Extent (PE) in LVM

1 What is a Physical Extent (PE)?

In **Linux Logical Volume Manager (LVM)**, a **Physical Extent (PE)** is the **smallest unit of storage allocation inside a Physical Volume (PV)**.

When a disk or partition is converted into an LVM **Physical Volume**, LVM divides that space into **equal-sized chunks called Physical Extents**.

Think of PE as **blocks of storage that LVM uses to build logical volumes**.

Hierarchy in LVM

Disk / Partition
↓
Physical Volume (PV)
↓
Physical Extents (PE)
↓
Volume Group (VG)
↓
Logical Extents (LE)

↓
Logical Volume (LV)
↓
Filesystem

So:

- **PE → belongs to PV**
 - **LE → belongs to LV**
 - **1 LE maps to 1 PE**
-

2 Why Physical Extents Are Important

Physical Extents make LVM **flexible and dynamic**.

Without PE:

- Storage would behave like traditional partitions (fixed size).

With PE:

- Storage can be **extended**
- Storage can be **moved between disks**
- Storage can be **combined from multiple disks**

Example:

Disk1 (100GB)

Disk2 (100GB)

Both become PVs

Combined into VG

VG = 200GB

LVM allocates **PE blocks from anywhere inside the VG**.

3 Default Physical Extent Size

By default, **LVM uses 4 MB PE size.**

This means:

1 PE = 4 MB

Example:

If a disk is **20 GB**:

20 GB / 4 MB = 5120 PE

So the PV will contain **5120 physical extents.**

4 Relationship Between PE and LE

Inside LVM:

Component	Meaning
PE	Physical Extent (on disk)
LE	Logical Extent (used by LV)

Mapping rule

1 LE = 1 PE

Example:

If you create a **10 GB Logical Volume**

PE size = 4MB

10GB / 4MB = 2560 extents

So:

LV uses 2560 LE

mapped to
2560 PE

5 Example Layout

Suppose we have:

Disk1 → 50GB
Disk2 → 50GB
PE size → 4MB

Convert disks to PV:

```
pvccreate /dev/sdb  
pvccreate /dev/sdc
```

Create VG:

```
vgcreate vg_data /dev/sdb /dev/sdc
```

Now:

VG size = 100GB

PE calculation:

$100\text{GB} / 4\text{MB} = 25600 \text{ PE}$

If you create a **20GB LV**

$20\text{GB} / 4\text{MB} = 5120 \text{ extents}$

LVM may allocate those **5120 PE from both disks**.

Example:

Disk1 → 3000 PE
Disk2 → 2120 PE

6 Viewing Physical Extents

6.1 Check PV details

`pvdisplay`

Example output:

```
PV Name      /dev/sdb
VG Name      vg_data
PV Size      50.00 GiB
PE Size      4.00 MiB
Total PE     12800
Free PE      8000
Allocated PE 4800
```

Explanation:

Field	Meaning
PE Size	Size of each extent
Total PE	Total extents in PV
Free PE	Available extents
Allocated PE	Used extents

6.2 View with `pvs`

`pvs`

Example:

```
PV      VG      Fmt Attr PSize  PFree
/dev/sdb vgdata lvm2 a-- 50.00g 10.00g
/dev/sdc vgdata lvm2 a-- 50.00g 20.00g
```

How LVM Uses Physical Extents

When you create an LV:

```
lvcreate -L 10G -n lv_data vg_data
```

LVM:

1. Calculates required extents
2. Allocates PE blocks from VG
3. Maps them to LE

Example mapping:

LV EXTENTS → PHYSICAL EXTENTS

LE1 → /dev/sdb PE1

LE2 → /dev/sdb PE2

LE3 → /dev/sdc PE1

LE4 → /dev/sdc PE2

Thus **data can span multiple disks**.

8 PE Size Configuration

PE size is set **when creating a Volume Group**.

Example:

```
vgcreate -s 8M vg_data /dev/sdb
```

Now:

PE size = 8MB

8.1 Why Change PE Size?

Larger PE size:

Advantages

- Supports **very large disks**
- Reduces **metadata overhead**

Disadvantages

- Less flexible resizing

Example:

PE Size	Smallest LV growth
4MB	4MB
16MB	16MB
64MB	64MB

9 How PE Helps in Extending Storage

Example scenario:

Initial LV:

LV = 10GB

PE size = 4MB

Required PE:

$10\text{GB} / 4\text{MB} = 2560 \text{ PE}$

Later extend LV:

`lvextend -L +5G /dev/vg_data/lv_data`

Extra PE required:

$5\text{GB} / 4\text{MB} = 1280 \text{ PE}$

LVM simply allocates **1280 free PE from VG**.

No repartitioning needed.

10 PE in Multiple Disk Allocation

Example VG:

Disk1 = 40GB
Disk2 = 60GB
Disk3 = 100GB

Total VG:

200GB

LVM does **not care which disk** PE comes from.

It allocates extents dynamically.

Example:

LV1 → Disk1 + Disk2

LV2 → Disk2 + Disk3

LV3 → Disk1 + Disk3

Visual Representation

Disk1 (/dev/sdb)

PE1 | PE2 | PE3 | PE4 | PE5 ...

Disk2 (/dev/sdc)

PE1 | PE2 | PE3 | PE4 | PE5 ...

Volume Group

All PE combined

Logical Volume

LE1 → sdb PE1

LE2 → sdb PE2

LE3 → sdc PE1

LE4 → sdc PE2

Advantages of Physical Extents (PE)

1. **Flexible storage management** – Disk space is divided into small chunks, making allocation easier.
 2. **Easy volume expansion** – Logical volumes can grow by adding more PE.
 3. **Combine multiple disks** – Storage from different disks can be pooled together.
 4. **Supports snapshots and data migration** – Enables backup and moving data between disks easily.
-

13 Disadvantages of Physical Extents (PE)

1. **Space wastage** – Large PE sizes can cause unused space (internal fragmentation).
 2. **Less flexibility with large PE** – Smallest allocation equals the PE size.
 3. **Not efficient for small volumes** – Large PE may waste space when creating small logical volumes.
 4. **Resizing limitations** – Volume size changes must be in multiples of PE size.
-

14. Hands-on Example of Physical Extent (PE)

Assume we have a new disk:

/dev/sdb

Disk size:

1 GB

Step 1: Create a Physical Volume

Convert the disk into a **Physical Volume (PV)**.

```
sudo pvcreate /dev/sdb
```

Verify:

```
pvs
```

Example output:

PV	VG	Fmt	Attr	PSize	PFree
/dev/sdb		lvm2	---	1.00g	1.00g

Step 2: Create a Volume Group with Custom PE Size

Now create a **Volume Group** and set the PE size.

Example: **PE size = 8MB**

```
sudo vgcreate -s 8M vg_test /dev/sdb
```

Here:

-s 8M → Physical Extent size = 8MB

vg_test → Volume group name

/dev/sdb → Physical volume

Step 3: Verify PE Size

Run:

```
vgdisplay vg_test
```

Example output:

VG Name	vg_test
VG Size	1.00 GiB

PE Size 8.00 MiB

Total PE 128

Free PE 128

Explanation:

$1\text{GB} = 1024\text{MB}$

$1024 / 8 = 128 \text{ PE}$

So the disk is divided into:

[PE1][PE2][PE3][PE4]...[PE128]

Step 4: Create Logical Volume Using PE

Create a **100MB** logical volume.

```
sudo lvcreate -L 100M -n lv_test vg_test
```

Verify:

```
lvdisplay
```

Example output:

LV Size 100.00 MiB

Current LE 13

Calculation:

PE size = 8MB

$100\text{MB} / 8\text{MB} \approx 12.5$

LVM rounds up:

13 PE allocated

Actual space used:

$13 \times 8\text{MB} = 104\text{MB}$

So **4MB** is unused inside the extent.

This is called **internal fragmentation**.

Step 5: Extend the Logical Volume

Increase the LV by **16MB**.

```
sudo lvextend -L +16M /dev/vg_test/lv_test
```

Calculation:

$16\text{MB} / 8\text{MB} = 2 \text{ PE}$

LVM adds **2 more PE**.

Now:

Total PE = 15

Visualization

Disk (/dev/sdb)

[PE1][PE2][PE3][PE4]...[PE128]

Logical Volume lv_test

[PE1][PE2][PE3]...[PE15]

Important Rules of Physical Extents

Rule 1: PE Size is Set During VG Creation

PE size is defined when creating the **Volume Group**.

Example:

```
vgcreate -s 8M vg_test /dev/sdb
```

Default:

4 MB

Rule 2: PE Size Cannot Be Changed Later

Once a VG is created:

PE size cannot be modified

To change PE size you must:

Delete VG → recreate VG

Rule 3: Logical Volume Size Must Be Multiple of PE

Example:

PE size = 8MB

LV sizes possible:

8MB

16MB

24MB

32MB

Not possible:

10MB

15MB

Rule 4: 1 Logical Extent = 1 Physical Extent

Mapping rule:

1 LE = 1 PE

Logical volumes use **Logical Extents**, which map to **Physical Extents**.

Rule 5: PE is the Smallest Allocation Unit in LVM

LVM allocates storage **only in PE units**.

Example:

PE size = 8MB

Smallest extension:

+8MB

Quick Summary

Concept	Explanation
PE	Smallest storage block in LVM
Default PE size	4 MB
Set PE size	During VG creation

Change PE size Not possible after VG creation

Allocation Done in multiples of PE

15. Swap Partition in Linux

1 What is Swap Partition?

A **Swap Partition** is a portion of the **hard disk used as virtual memory** when the system's **RAM becomes full**.

In simple terms:

👉 **Swap = Backup memory for RAM**

When RAM runs out of space, Linux moves **inactive data from RAM to swap space** to free RAM for active processes.

2 Why Swap is Needed

RAM is **fast but limited**.

Swap helps the system when memory demand increases.

Benefits:

- Prevents system crash when RAM is full
 - Allows more programs to run simultaneously
 - Supports **hibernation** (saving RAM contents to disk)
-

3 How Swap Works

Example:

System configuration:

RAM = 4GB

Swap = 2GB

Scenario:

1. Applications start using RAM
2. RAM becomes almost full
3. Linux moves **less-used processes to swap**
4. RAM becomes free for active processes

Flow:

Program → RAM → If RAM Full → Move to Swap (Disk)

4 Types of Swap in Linux

1 Swap Partition

A **dedicated disk partition** used as swap.

Example:

/dev/sda2 → swap

Advantages:

- Faster than swap file
 - More stable
-

2 Swap File

A **regular file** used as swap space.

Example:

/swapfile

Advantages:

- Easy to create and resize
- No need to repartition disk

5 How to Check Swap in Linux

Check swap usage:

`free -h`

Example output:

	total	used	free
Mem:	8G	3G	5G
Swap:	2G	0B	2G

Check swap devices:

`swapon --show`

Example:

NAME	TYPE	SIZE	USED
/dev/sda2	partition	2G	0B

6 How to Create a Swap Partition

Step 1: Create partition using `fdisk`


```
fdisk /dev/sdb
```

Set type:

82 → Linux Swap

Step 2: Format as swap

```
mkswap /dev/sdb1
```

Step 3: Enable swap

```
swapon /dev/sdb1
```

Step 4: Make it permanent

Edit `/etc/fstab`

```
/dev/sdb1 swap swap defaults 0 0
```

7 How to Create Swap File

Create 1GB swap file:

```
fallocate -l 1G /swapfile
```

Set permission:

```
chmod 600 /swapfile
```

Create swap:

```
mkswap /swapfile
```

Enable swap:

```
swapon /swapfile
```

8 How to Disable Swap

Temporarily disable swap:

```
swapoff -a
```

Enable again:

```
swapon -a
```

9 Recommended Swap Size (General Rule)

RAM	Recommended Swap
2GB	2GB
4GB	2–4GB
8GB	4GB
16GB+	4–8GB

For **hibernation**, swap should be **equal to RAM**.

10 Important Swap Commands

Command	Purpose
<code>free -h</code>	Check memory usage

`swapon --show` Show swap devices

`mkswap` Format swap space

`swapon` Enable swap

`swapoff` Disable swap

Quick Summary

- Swap is **disk space used as virtual memory**
- Used when **RAM becomes full**
- Two types: **Swap Partition** and **Swap File**
- Helps system stability and multitasking

Simple Definition

Swap partition is a disk space used by Linux as **extra memory when RAM is full**.

Real-Time Scenario

Step 1: Install EPEL Repository

Run:

```
yum install epel-release -y
```

or

```
dnf install epel-release -y
```

Verify:

```
dnf repolist
```

You should see:

```
epel
```

Step 2: Install Stress Tool

Now install stress:

```
dnf install stress -y
```

Check installation:

```
stress --version
```

Step 3: Run Stress Test (Memory)

Example:

```
stress --vm 2 --vm-bytes 512M --timeout 60s
```

Meaning:

Option	Purpose
<code>--vm 2</code>	create 2 memory workers

`--vm-bytes 512M` each worker uses 512MB RAM

`--timeout 60s` run for 60 seconds

This will **force RAM usage and trigger swap**.

Step 4: Monitor Swap Usage

Open another terminal and run:

watch free -h or top

You will see **swap usage increasing**.
